



Stephan Schwarz

Lasttests für Kubernetes-Workloads

about me

44 Jahre alt

Verheiratet, zwei Kinder

Rollen in der Vergangenheit:

- Network Administrator
- Systems Engineer
- Cloud Engineer
- Infrastructure Developer
- Platform Engineer

seit 2017: Kubernetes

seit 2021: Consultant



Stephan Schwarz

Senior DevOps Engineer / Consultant

iits-consulting GmbH

what about you?



Es war einmal vor langer Zeit in einer
weit, weit entfernten Galaxis...



Was ist Performance?

Performant = **Erwartungen** werden **erfüllt** an:

- Durchsatz
- Latenz
- Skalierbarkeit
- Ressourcenverbrauch

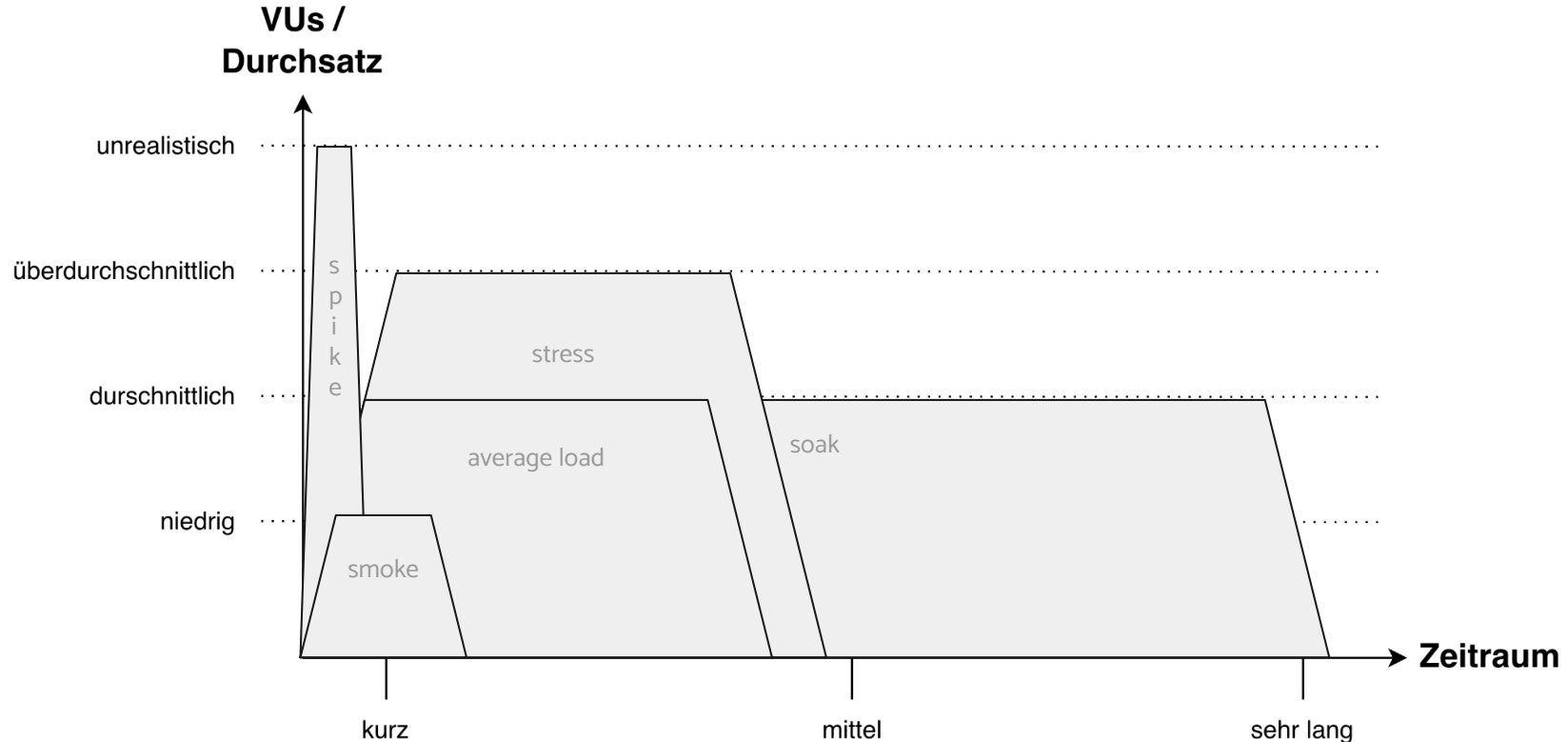


Warum Lasttests?

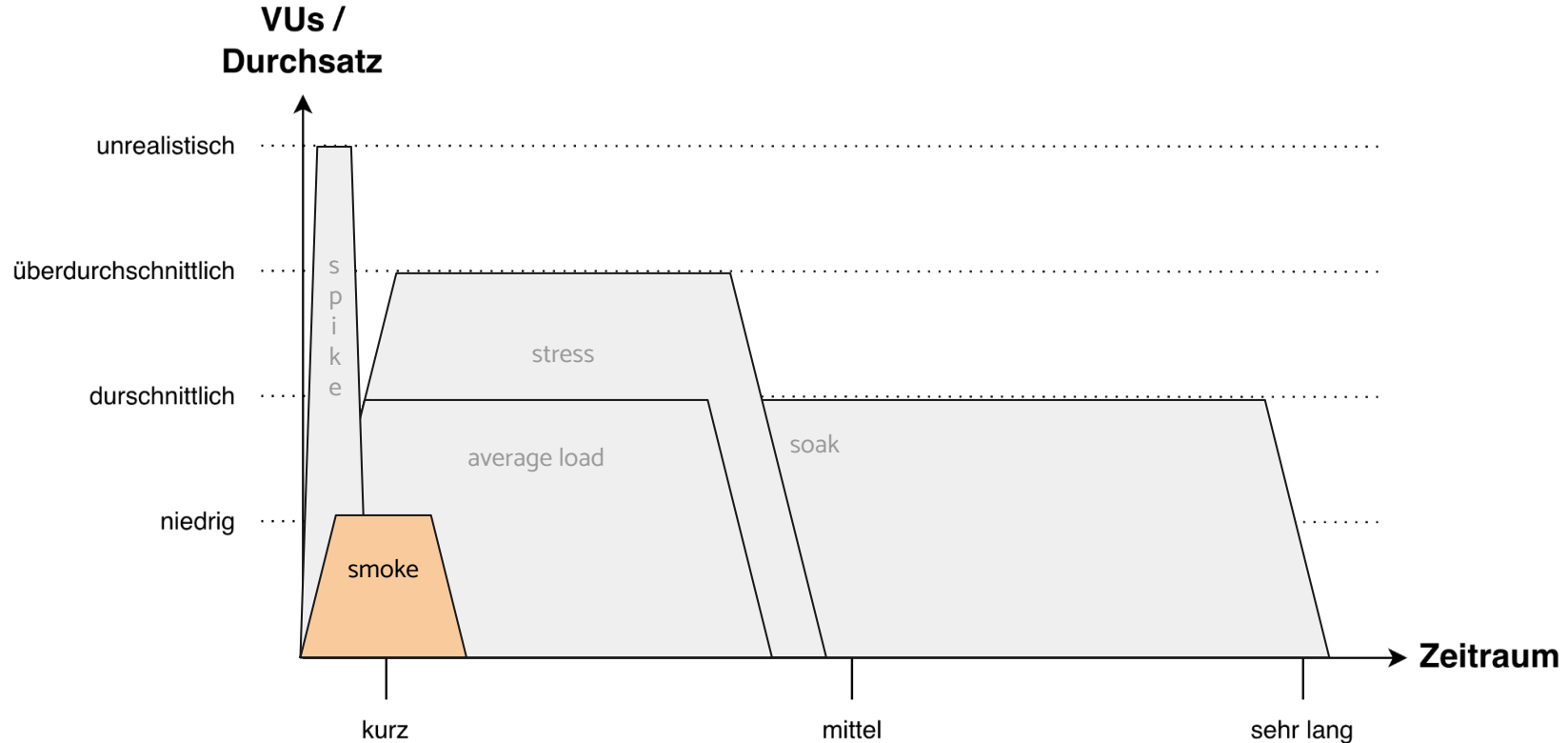
- Aufdecken von **Bottlenecks**
- **Stabiler Betrieb**
- Testen der **Skalierbarkeit**
- Sicherstellung von **SLAs**
- Ermittlung der benötigten **Ressourcen**
- Optimierung von **Kosten**



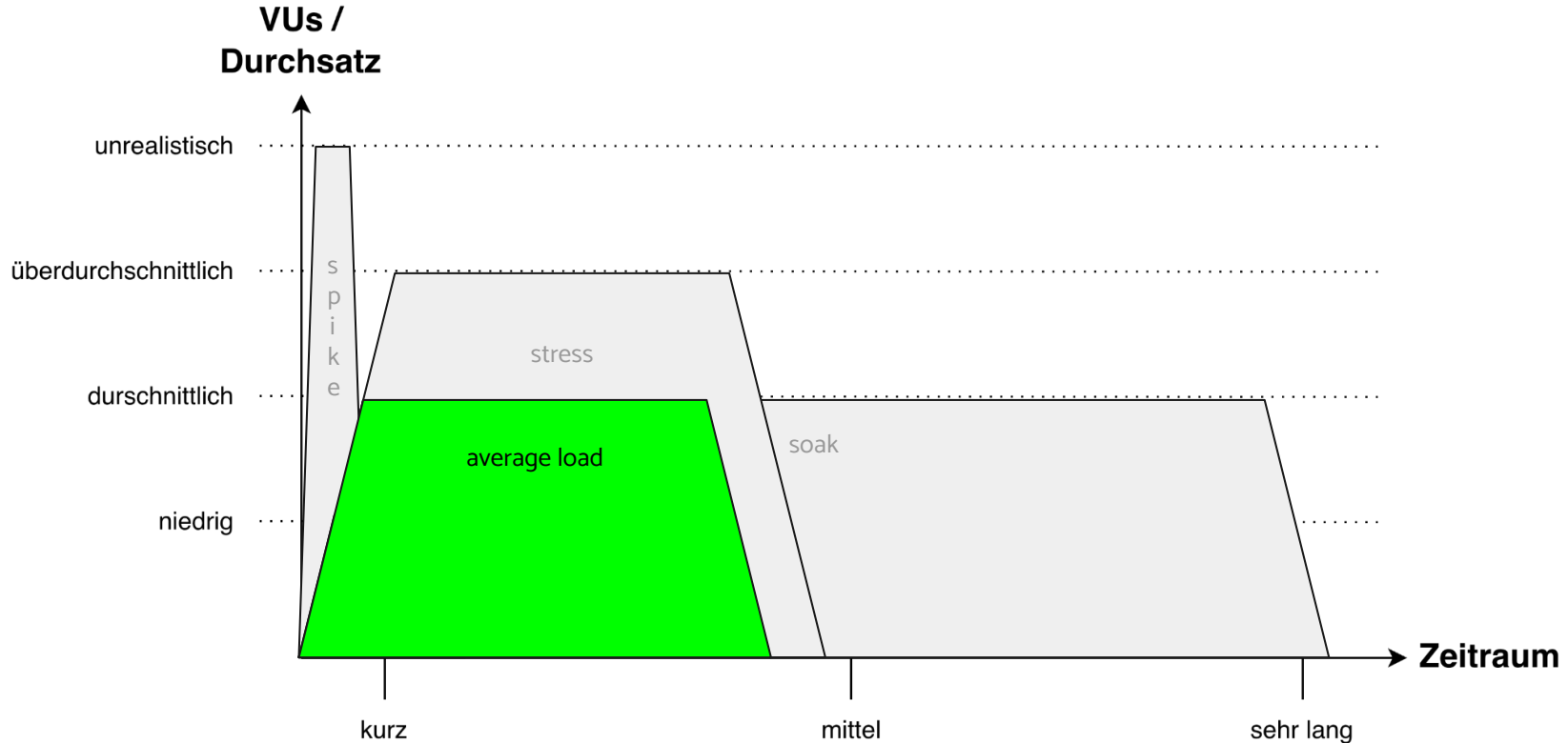
Arten von Lasttests



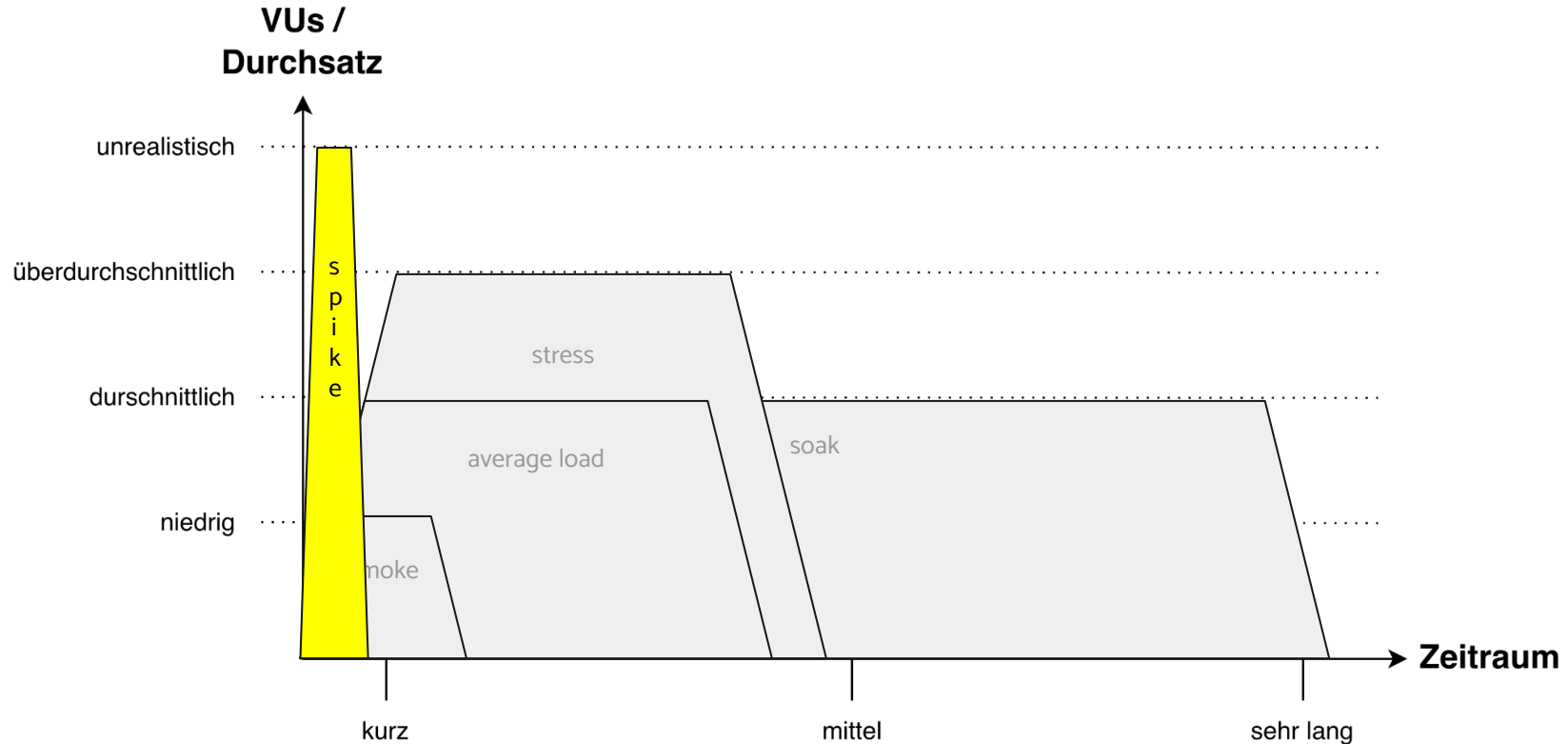
Smoke Test



Average Load Test



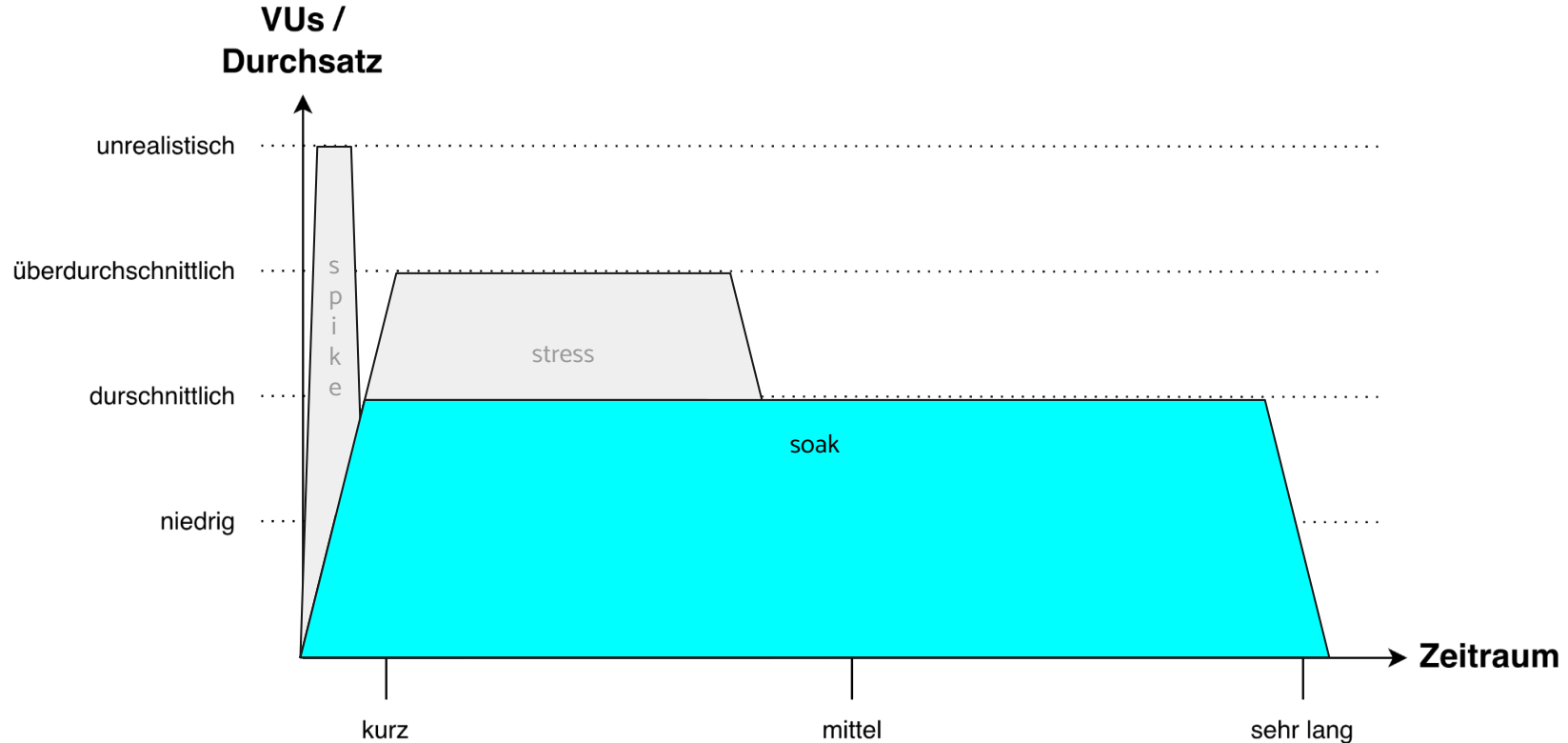
Spike Test



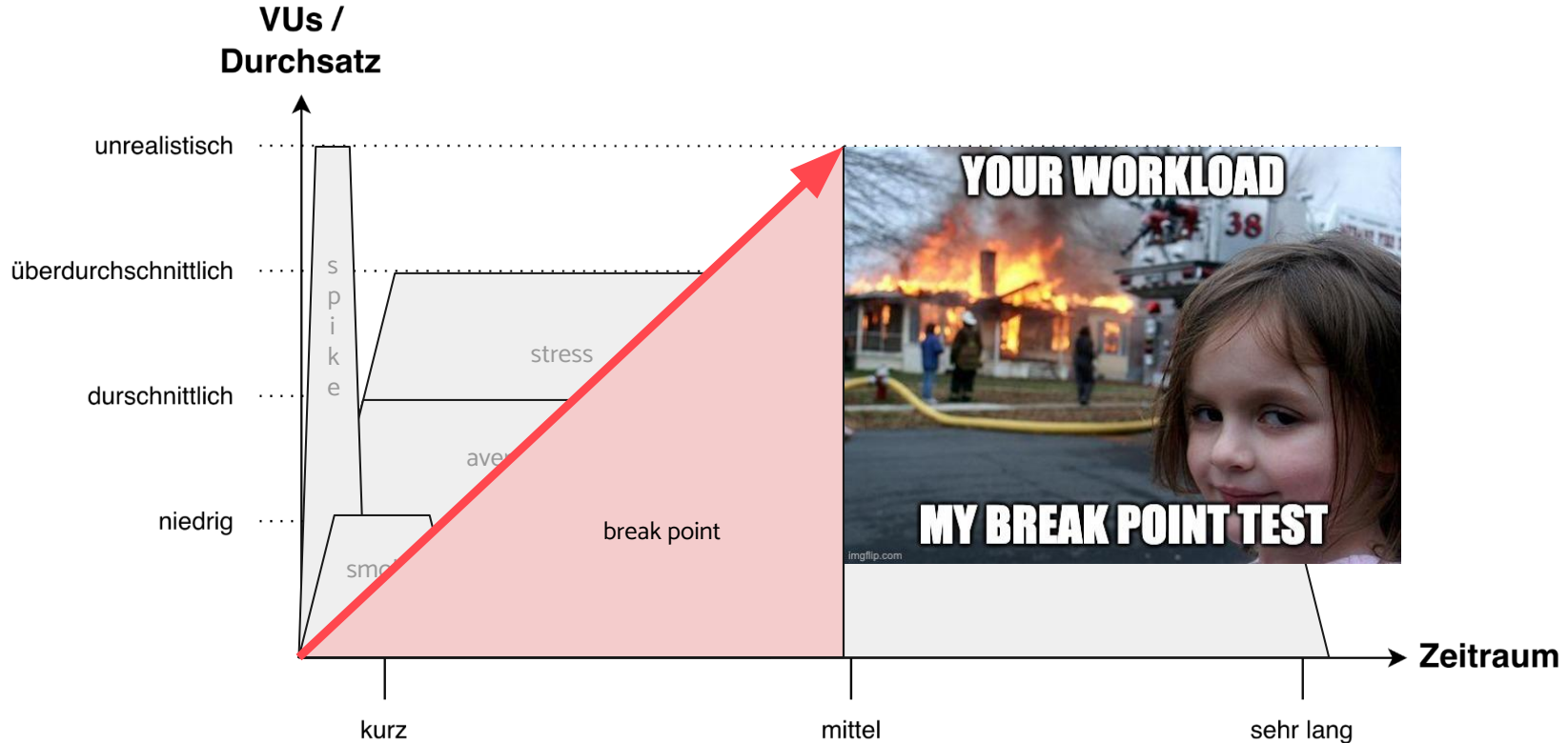
Stress Test

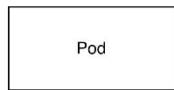


Soak Test



Break Point Test

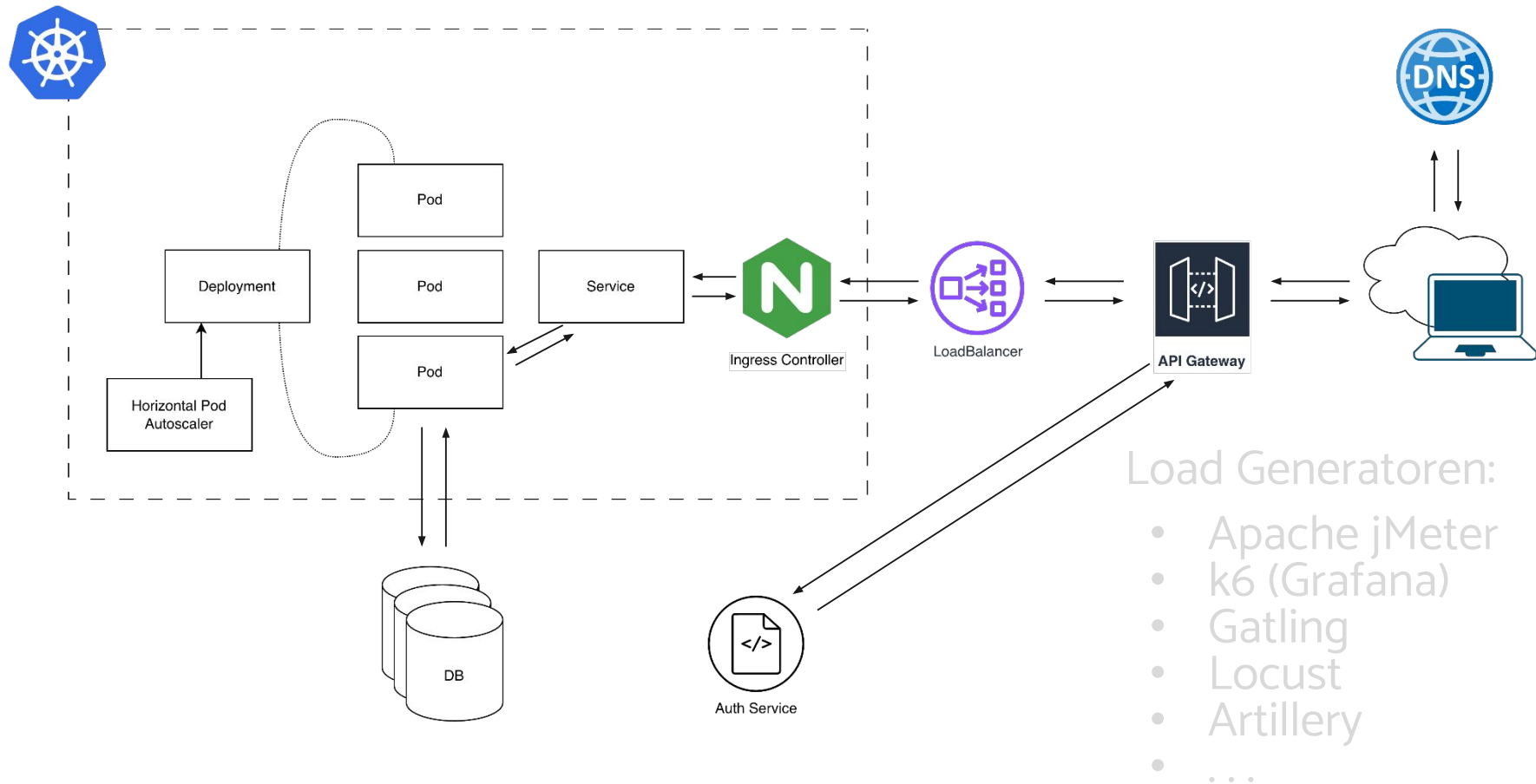




Load Generatoren:

- Apache jMeter
- k6 (Grafana)
- Gatling
- Locust
- Artillery
- ...

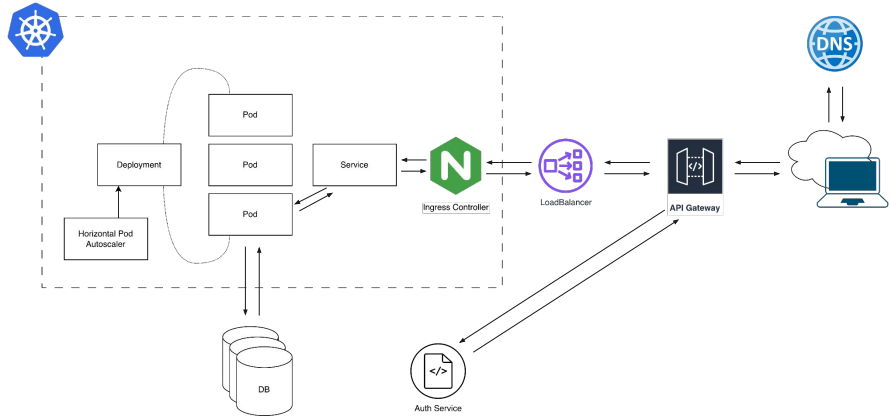




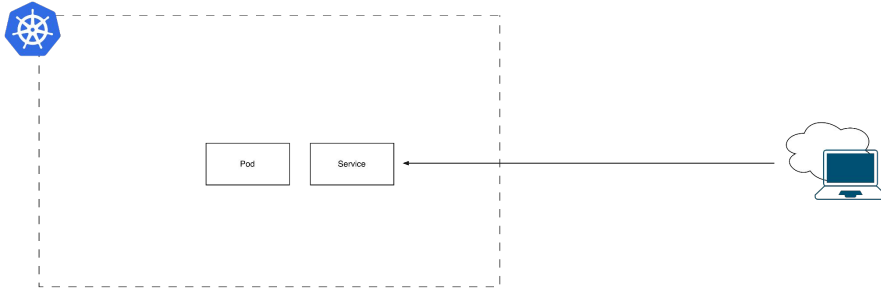
Was wird getestet?

End-to-end

- Realitätsnah
- Kundenperspektive
- Testet komplette Infrastruktur
- Abhängige Services werden mitgetestet



Was wird getestet?



Labor Test

- z.B. **Single Pod** Perspektive
- Ermittlung der **Resource Usage**
- Experimentieren mit Werten für **requests** und **limits**
- Abhängige Services und Infrastruktur werden nur bei Notwendigkeit mitgetestet



Wie funktioniert ein Load Generator?

JS basic-load.js > ...

```
1 import { check, sleep } from "k6";  
2 import http from "k6/http";  
3
```

Wie funktioniert ein Load Generator?

Stages:

- Ramp-up
- Main test
- Ramp-down

Target Virtual Users:

- Parallele Ausführungen

```
JS basic-load.js > ...
1  import { check, sleep } from "k6";
2  import http from "k6/http";
3
4  export let options = {
5    stages: [
6      { duration: "1m", target: 5 },
7      { duration: "3m", target: 10 },
8      { duration: "1m", target: 0 },
9    ]
10 };
11
```

Wie funktioniert ein Load Generator?

Stages:

- Ramp-up
- Main test
- Ramp-down

Target Virtual Users:

- Parallele Ausführungen

Die eigentlichen Tests

JS basic-load.js > ...

```
1  import { check, sleep } from "k6";
2  import http from "k6/http";
3
4  export let options = {
5    stages: [
6      { duration: "1m", target: 5 },
7      { duration: "3m", target: 10 },
8      { duration: "1m", target: 0 },
9    ]
10 };
11
12 export default function() {
13   let res = http.get("https://iits-consulting.de");
14   check(res, {
15     "is status 200": (r) => r.status === 200
16   });
17   sleep(1);
18 };
19
```

❌ k6 run ./basic-load.js



execution: local
script: ./basic-load.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 5m30s max duration (incl. graceful stop):
* default: Up to 10 looping VUs for 5m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

running (0m44.5s), 03/10 VUs, 73 complete and 0 interrupted iterations
default [====>-----] 03/10 VUs 0m44.5s/5m00.0s

⊗ k6 run ./basic-load.js



```
execution: local
  script: ./basic-load.js
  output: -
```

scenarios: (100.00%) 1 scenario, 10 max VUs, 5m30s max duration (incl. graceful stop):

* default: Up to 10 looping VUs for 5m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

running (0m44.5s), 03/10 VUs, 73 complete and 0 interrupted iterations

default [====>-----] 03/10 VUs 0m44.5s/5m00.0s

⊗ k6 run ./basic-load.js



execution: local
script: ./basic-load.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 5m30s max duration (incl. graceful stop):
* default: Up to 10 looping VUs for 5m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

running (0m44.5s), 03/10 VUs, 73 complete and 0 interrupted iterations
default [====>-----] 03/10 VUs 0m44.5s/5m00.0s

⊗ k6 run ./basic-load.js



execution: local
script: ./basic-load.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 5m30s max duration (incl. graceful stop):
* default: Up to 10 looping VUs for 5m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

running (0m44.5s), 03/10 VUs, 73 complete and 0 interrupted iterations
default [====>-----] 03/10 VUs 0m44.5s/5m00.0s

TOTAL RESULTS

```
checks_total.....: 1543    5.127954/s
checks_succeeded...: 71.35% 1101 out of 1543
checks_failed.....: 28.64% 442 out of 1543
```

```
x is status 200
↳ 71% - ✓ 1101 / x 442
```

HTTP

```
http_req_duration.....: avg=130.43ms min=22.99ms  med=148.97ms max=1.27s p(90)=180.94ms p(95)=188.84ms
{ expected_response:true }...: avg=165.55ms min=120.53ms med=157.05ms max=1.27s p(90)=184.55ms p(95)=192.75ms
http_req_failed.....: 28.64% 442 out of 1543
http_reqs.....: 1543    5.127954/s
```

EXECUTION

```
iteration_duration.....: avg=1.13s    min=1.02s    med=1.15s    max=2.28s p(90)=1.18s    p(95)=1.19s
iterations.....: 1543    5.127954/s
vus.....: 1      min=1      max=10
vus_max.....: 10     min=10     max=10
```

NETWORK

```
data_received.....: 203 MB 675 kB/s
data_sent.....: 771 kB 2.6 kB/s
```

TOTAL RESULTS

```
checks_total.....: 1543    5.127954/s
checks_succeeded...: 71.35% 1101 out of 1543
checks_failed.....: 28.64% 442 out of 1543
```

```
x is status 200
↳ 71% - ✓ 1101 / x 442
```

HTTP

```
http_req_duration.....: avg=130.43ms min=22.99ms  med=148.97ms max=1.27s p(90)=180.94ms p(95)=188.84ms
{ expected_response:true }...: avg=165.55ms min=120.53ms med=157.05ms max=1.27s p(90)=184.55ms p(95)=192.75ms
http_req_failed.....: 28.64% 442 out of 1543
http_reqs.....: 1543    5.127954/s
```

EXECUTION

```
iteration_duration.....: avg=1.13s    min=1.02s    med=1.15s    max=2.28s p(90)=1.18s    p(95)=1.19s
iterations.....: 1543    5.127954/s
vus.....: 1      min=1      max=10
vus_max.....: 10     min=10     max=10
```

NETWORK

```
data_received.....: 203 MB 675 kB/s
data_sent.....: 771 kB 2.6 kB/s
```

TOTAL RESULTS

```
checks_total.....: 1543    5.127954/s
checks_succeeded...: 71.35% 1101 out of 1543
checks_failed.....: 28.64% 442 out of 1543
```

```
x is status 200
↳ 71% - ✓ 1101 / x 442
```

HTTP

```
http_req_duration.....: avg=130.43ms min=22.99ms med=148.97ms max=1.27s p(90)=180.94ms p(95)=188.84ms
{ expected_response:true }...: avg=165.55ms min=120.53ms med=157.05ms max=1.27s p(90)=184.55ms p(95)=192.75ms
http_req_failed.....: 28.64% 442 out of 1543
http_reqs.....: 1543    5.127954/s
```

EXECUTION

```
iteration_duration.....: avg=1.13s    min=1.02s    med=1.15s    max=2.28s p(90)=1.18s    p(95)=1.19s
iterations.....: 1543    5.127954/s
vus.....: 1    min=1    max=10
vus_max.....: 10    min=10    max=10
```

NETWORK

```
data_received.....: 203 MB 675 kB/s
data_sent.....: 771 kB 2.6 kB/s
```

TOTAL RESULTS

checks_total.....: 1543 5.127954/s
checks_succeeded...: 71.35% 1101 out of 1543
checks_failed.....: 28.64% 442 out of 1543

x is status 200
↳ 71% - ✓ 1101 / x 442

HTTP

http_req_duration.....: avg=130.43ms min=22.99ms med=148.97ms max=1.27s p(90)=180.94ms p(95)=188.84ms
{ expected_response:true }...: avg=165.55ms min=120.53ms med=157.05ms max=1.27s p(90)=184.55ms p(95)=192.75ms
http_req_failed.....: 28.64% 442 out of 1543
http_reqs.....: 1543 5.127954/s

EXECUTION

iteration_duration.....: avg=1.13s min=1.02s med=1.15s max=2.28s p(90)=1.18s p(95)=1.19s
iterations.....: 1543 5.127954/s
vus.....: 1 min=1 max=10
vus_max.....: 10 min=10 max=10

NETWORK

data_received.....: 203 MB 675 kB/s
data_sent.....: 771 kB 2.6 kB/s

TOTAL RESULTS

```
checks_total.....: 1543    5.127954/s
checks_succeeded...: 71.35% 1101 out of 1543
checks_failed.....: 28.64% 442 out of 1543
```

```
x is status 200
↳ 71% - ✓ 1101 / x 442
```

HTTP

```
http_req_duration.....: avg=130.43ms min=22.99ms med=148.97ms max=1.27s p(90)=180.94ms p(95)=188.84ms
{ expected_response:true }...: avg=165.55ms min=120.53ms med=157.05ms max=1.27s p(90)=184.55ms p(95)=192.75ms
http_req_failed.....: 28.64% 442 out of 1543
http_reqs.....: 1543    5.127954/s
```

EXECUTION

```
iteration_duration.....: avg=1.13s    min=1.02s    med=1.15s    max=2.28s p(90)=1.18s    p(95)=1.19s
iterations.....: 1543    5.127954/s
vus.....: 1    min=1    max=10
vus_max.....: 10    min=10    max=10
```

NETWORK

```
data_received.....: 203 MB 675 kB/s
data_sent.....: 771 kB 2.6 kB/s
```


TOTAL RESULTS

```
checks_total.....: 1543    5.127954/s
checks_succeeded...: 71.35% 1101 out of 1543
checks_failed.....: 28.64% 442 out of 1543
```

```
x is status 200
↳ 71% - ✓ 1101 / x 442
```

HTTP

```
http_req_duration.....: avg=130.43ms min=22.99ms  med=148.97ms max=1.27s p(90)=180.94ms p(95)=188.84ms
{ expected_response:true }...: avg=165.55ms min=120.53ms med=157.05ms max=1.27s p(90)=184.55ms p(95)=192.75ms
http_req_failed.....: 28.64% 442 out of 1543
http_reqs.....: 1543    5.127954/s
```

EXECUTION

```
iteration_duration.....: avg=1.13s    min=1.02s    med=1.15s    max=2.28s p(90)=1.18s    p(95)=1.19s
iterations.....: 1543    5.127954/s
vus.....: 1      min=1      max=10
vus_max.....: 10     min=10     max=10
```

NETWORK

```
data_received.....: 203 MB 675 kB/s
data_sent.....: 771 kB 2.6 kB/s
```

PERFORMANCE OVERVIEW



The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.



REQUESTS MADE

1.3K reqs

HTTP FAILURES

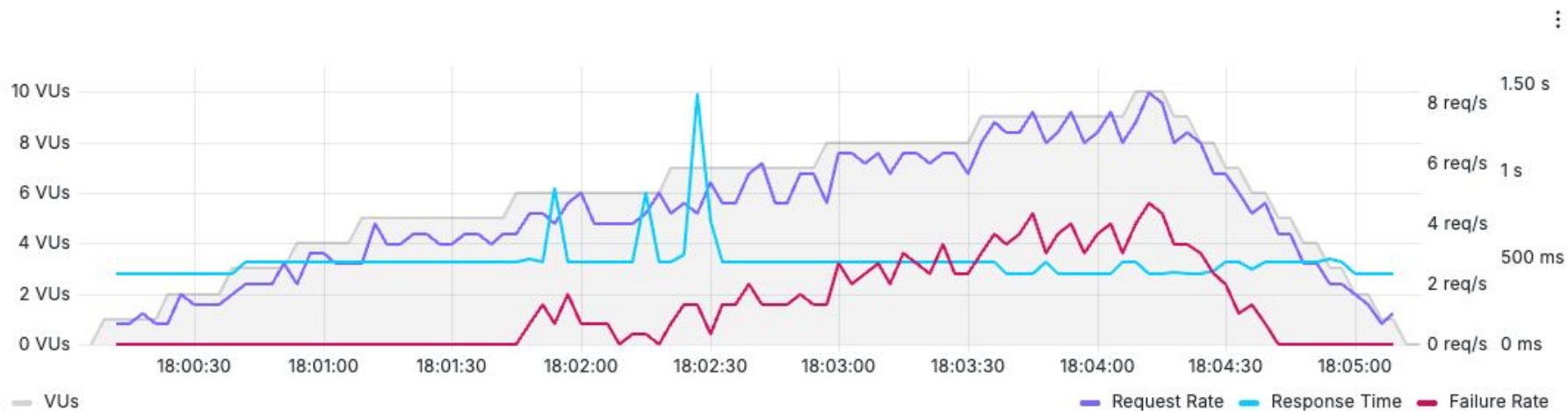
371 reqs

PEAK RPS

8.33 reqs/s

P95 RESPONSE TIME

477 ms



SCENARIOS (0)

SHOW



No scenario was configured in this test. [Check out how to create one](#)

PERFORMANCE OVERVIEW



The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.



REQUESTS MADE

1.3K reqs

HTTP FAILURES

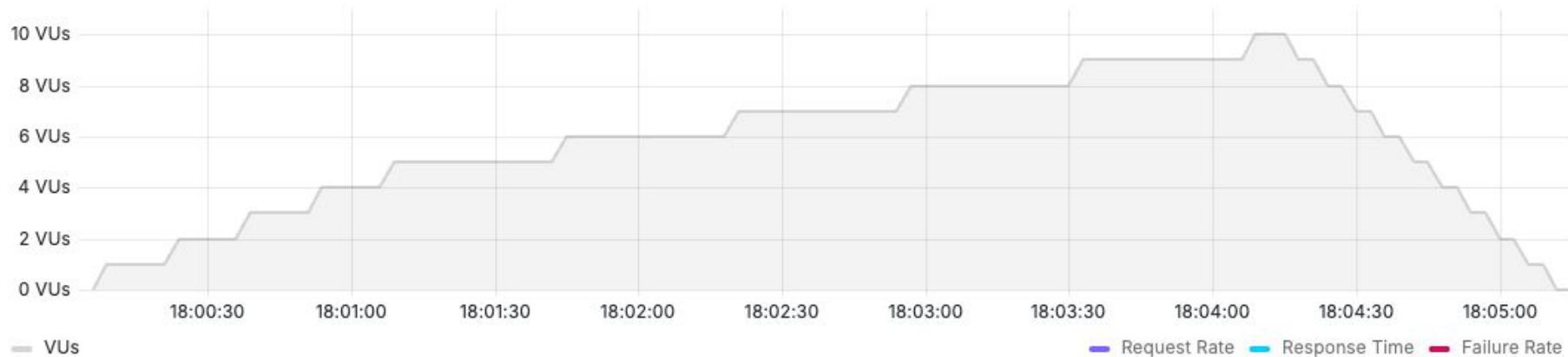
371 reqs

PEAK RPS

8.33 reqs/s

P95 RESPONSE TIME

477 ms



SCENARIOS (0)

SHOW



No scenario was configured in this test. [Check out how to create one](#)

PERFORMANCE OVERVIEW



The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.



REQUESTS MADE

1.3K reqs

HTTP FAILURES

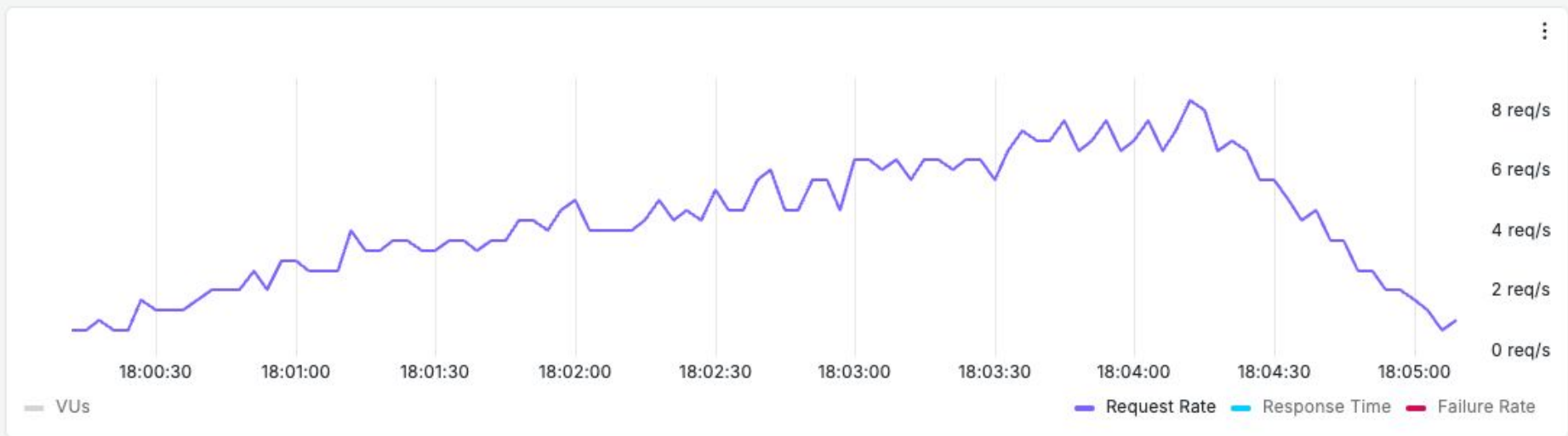
371 reqs

PEAK RPS

8.33 reqs/s

P95 RESPONSE TIME

477 ms



SCENARIOS (0)

SHOW



No scenario was configured in this test. [Check out how to create one](#)

PERFORMANCE OVERVIEW



The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.



REQUESTS MADE

1.3K reqs

HTTP FAILURES

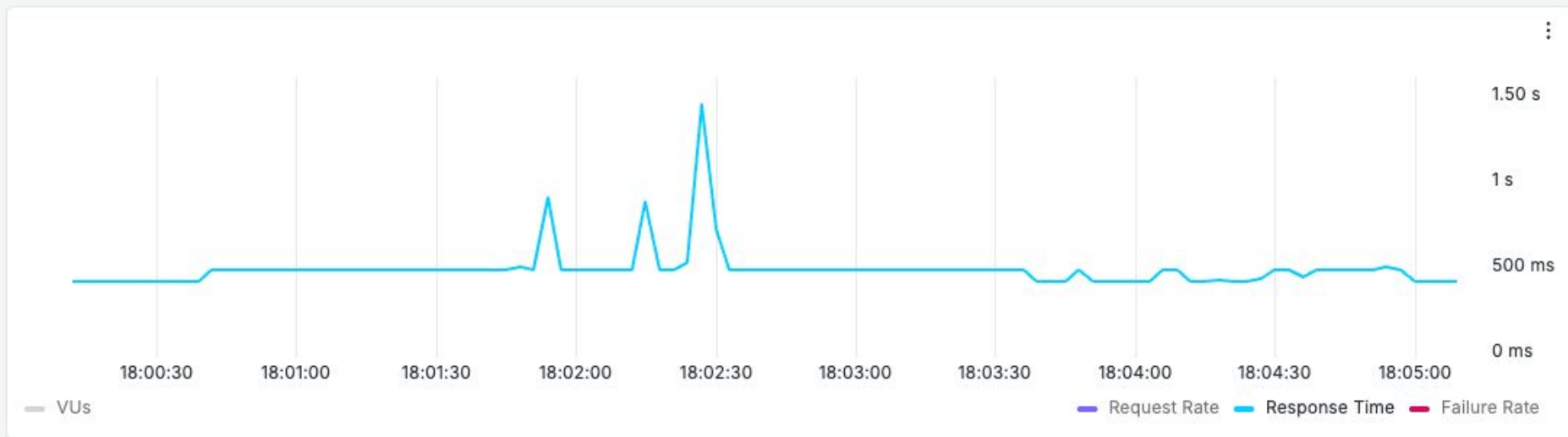
371 reqs

PEAK RPS

8.33 reqs/s

P95 RESPONSE TIME

477 ms



SCENARIOS (0)

SHOW



No scenario was configured in this test. [Check out how to create one](#)

PERFORMANCE OVERVIEW



The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.

REQUESTS MADE

1.3K reqs

HTTP FAILURES

371 reqs

PEAK RPS

8.33 reqs/s

P95 RESPONSE TIME

477 ms



SCENARIOS (0)

SHOW



No scenario was configured in this test. [Check out how to create one](#)

PERFORMANCE OVERVIEW



The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.

REQUESTS MADE

1.3K reqs

HTTP FAILURES

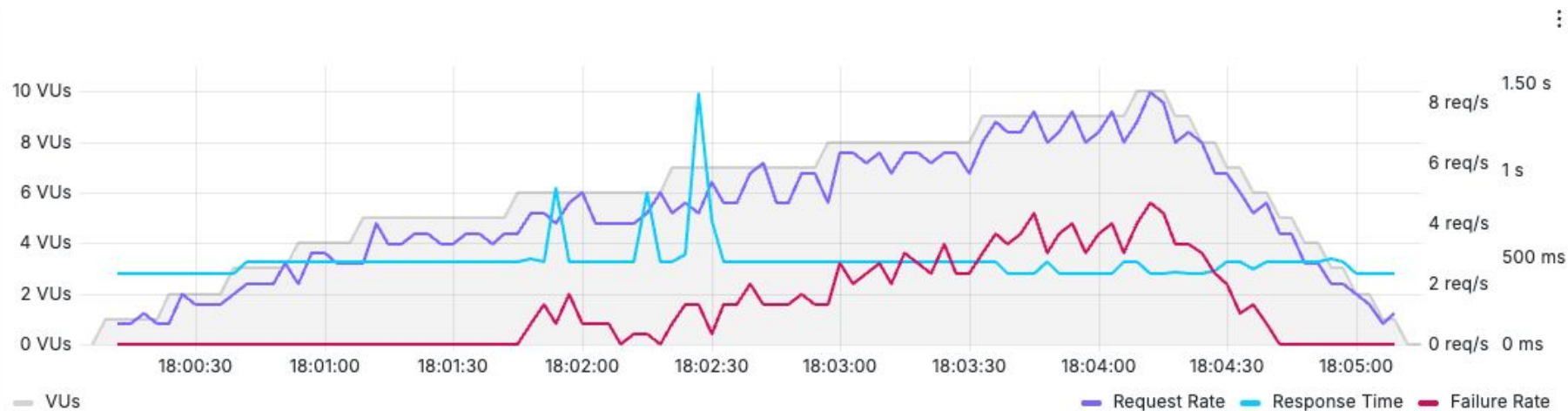
371 reqs

PEAK RPS

8.33 reqs/s

P95 RESPONSE TIME

477 ms



SCENARIOS (0)

SHOW

No scenario was configured in this test. [Check out how to create one](#)

Quick select ▾ Add filter ▾ List Tree										
⋮ URL >	SCENARIO >	METHOD >	STATUS >	COUNT >	MIN >	AVG >	STDDEV >	P95 >	P99 >	MAX >
✓ iits-consulting.de/	default	GET	200	932	404 ms	431 ms	92 ms	477 ms	493 ms	1 s
✗ iits-consulting.de/	default	GET	429	371	101 ms	113 ms	8.2 ms	119 ms	119 ms	119 ms

HTTP/1.1 429 Too Many Requests

Gelernte Lektionen (1 / 2)

Zugriff auf weitere Systeme notwendig:

- Monitoring
- Metriken
- Traces
- Logs



Gelernte Lektionen (2 / 2)

- Lasttest vorher **kommunizieren** (relevante Abteilungen / Dienstleister)
- Nie mehr als **eine Änderung** gleichzeitig
- Veränderung und Ergebnis **dokumentieren**
- Tests **wiederholen** um zufällige äußere Faktoren aufzudecken
- Wenn etwas bricht, **Ursache(n) finden**
- Dinge werden kaputt gehen

The background of the slide is a dark, starry space. In the upper right corner, a blue and white Earth is visible. Below it, a black spaceship with a blue engine glow is flying towards the left.

**Mögen die Lasttests
mit Ihnen sein...**



...und Ihnen Erkenntnisse bringen.



Vielen Dank

Stephan Schwarz
stephan.schwarz@iits-consulting.de

